

Тема 6. Соединение таблиц. Использование подзапросов

Одна из наиболее важных особенностей запросов SQL – это их способность определять связи между различными таблицами и выводить информацию из них в терминах этих связей.

Этот вид операции называется — **соединением**, которое является одним из видов операций в реляционных базах данных. При соединении, таблицы, представленные списком в предложении FROM запроса, отделяются запятыми. Предикат запроса может ссылаться к любому столбцу любой связанной таблицы и, следовательно, может использоваться для связи между ними. Обычно предикат сравнивает значения в столбцах различных таблиц, чтобы определить, удовлетворяет ли WHERE установленному условию.

Полное имя столбца таблицы фактически состоит из имени таблицы, сопровождаемого точкой и затем именем столбца. Имеются несколько примеров имен:

```
Salespeople.snum  
Salespeople.city  
Orders.odate
```

До этого, мы могли опускать имена таблиц, потому что запрашивали только одну таблицу. Даже когда мы делаем запрос из нескольких таблиц, можно опускать имена таблиц, если все столбцы имеют различные имена. Но так бывает не всегда. Например, мы имеем две типовые таблицы со столбцами называемыми city.

Если мы должны связать эти столбцы (кратковременно), мы будем должны указать их с именами Salespeople.city или Customers.city, чтобы SQL мог их различать.

Предположим, что надо поставить в соответствии продавцу его заказчиков в городе, в котором они живут, т.е. увидеть все комбинации продавцов и заказчиков из одного этого города. Для этого надо брать каждого продавца и искать в таблице Заказчиков всех заказчиков, расположенных в том же городе:

```
SELECT Customers.cname, Salespeople.sname, Salespeople.city  
FROM Salespeople, Customers  
WHERE Salespeople.city = Customers.city;
```

Так как поле city имеется и в таблице Продавцов, и таблице Заказчиков, имена таблиц должны использоваться как префиксы. Хотя это необходимо, только когда два или более полей имеют одно и то же имя. Мы будем в наших примерах далее использовать имена таблиц, только когда необходимо, так что будет ясно, когда они необходимы, а когда нет.

SQL в соединении исследует каждую комбинацию строк двух или более возможных таблиц, и проверяет эти комбинации по их предикатам.

Соединения, которые используют предикаты, основанные на равенствах называются — соединениями по равенству.

Соединения по равенству это наиболее общий вид соединения, но имеются и другие. Можно, фактически, использовать любой из реляционных операторов в соединении. Ниже показан пример другого вида соединения:

```
SELECT sname, cname  
FROM Salespeople, Customers  
WHERE sname < cname AND rating < 200;
```

Эта команда воспроизводит все комбинации имени продавца и имени заказчика так, что первый предшествует последнему в алфавитном порядке, а последний имеет оценку меньше чем 200.

Соединение более двух таблиц

Можно также создавать запросы, объединяющие более двух таблиц. Предположим, что надо найти все Заказы заказчиков не находящихся в тех городах где находятся их продавцы. Для этого необходимо связать все три таблицы:

```
SELECT onum, cname, Orders.cnum, Orders.snum  
FROM Salespeople, Customers, Orders  
WHERE Customers.city <> Salespeople.city  
AND Orders.cnum = Customers.cnum  
AND Orders.snum = Salespeople.snum;
```

Хотя эта команда выглядит скорее как комплексная, вы можете следовать за логикой, просто проверяя, что заказчики не размещены в тех городах, где размещены их продавцы (совпадение двух snum полей), и что перечисленные Заказы выполнены с помощью этих заказчиков (совпадение Заказов с полями snum и snum в таблице Заказов).

Соединение таблицы с собой

Для объединения таблицы с собой, можно сделать каждую строку таблицы, одновременно, и комбинацией ее с собой и комбинацией с каждой другой строкой таблицы. Вы затем оцениваете каждую комбинацию в терминах предиката, также как в объединениях разных таблиц. Это позволит вам легко создавать определенные виды связей между различными позициями внутри одиночной таблицы — с помощью обнаружения пар строк со значением поля, например.

Вы можете изобразить соединение таблицы с собой, как соединение двух копий одной и той же таблицы. Таблица на самом деле не копируется, но SQL выполняет команду так, как если бы это было сделано.

Псевдонимы

Синтаксис команды для соединения таблицы с собой тот же, что и для соединения разных таблиц в одном запросе.

Когда вы соединяете таблицу с собой, все повторяемые имена столбца, заполняются префиксами имени таблицы. Чтобы ссылаться к этим столбцам внутри запроса, вы должны иметь два различных имени для этой таблицы.

Это можно сделать с помощью определения временных имен, называемых переменными диапазона, переменными корреляции или просто псевдонимами.

Это определяется в предложении FROM запроса: набираете имя таблицы, оставляете пробел, и затем набираете псевдоним для нее.

Например, найти все пары заказчиков имеющих один и тот же рейтинг:

```
SELECT first.cname, second.cname, first.rating
FROM Customers first, Customers second
WHERE first.rating = second.rating;
```

В вышеупомянутой команде, SQL ведет себя так, как если бы он соединял две таблицы называемые 'первая' и 'вторая'. Обе они, фактически, таблицы Заказчиков, но псевдонимы разрешают им быть обработанными независимо. Псевдонимы первый и второй были установлены в предложении FROM запроса, сразу после имени копии таблицы.

Обратите внимание, что псевдонимы могут использоваться в предложении SELECT, даже если они не определены в предложении FROM.

Как работает подзапрос?

С помощью SQL вы можете вкладывать запросы внутрь друг друга. Обычно, внутренний запрос генерирует значение, которое проверяется в предикате внешнего запроса, определяющего, верно оно или нет.

Предположим что мы знаем имя sname продавца Motika, но не знаем его номер snum, и хотим извлечь все его Заказы из таблицы Заказов. Имеется один способ, чтобы сделать это (вывод показывается в Рисунке):

```
SELECT *
FROM Orders
WHERE snum = (SELECT snum
              FROM Salespeople
              WHERE sname = 'Motika');
```

Чтобы оценить внешний (основной) запрос, SQL сначала должен оценить внутренний запрос (или подзапрос) внутри предложения WHERE. Он делает это так, как и должен делать запрос, имеющий единственную цель – отыскать через таблицу Продавцов все строки, где поле sname равно значению Motika, и затем извлечь значения поля snum этих строк.

Единственной найденной строкой, естественно, будет snum = 1004. Однако SQL не просто выдает это значение, а помещает его в предикат основного запроса вместо самого подзапроса, так чтобы предиката прочитал, что snum = 1004

Основной запрос затем выполняется как обычно с вышеупомянутыми результатами. Конечно же, подзапрос должен выбрать один и только один столбец, а тип данных этого столбца должен совпадать с тем значением, с которым он будет сравниваться в предикате. Часто, как показано выше, выбранное поле и его значение будут иметь одинаковые имена (в этом случае, `snum`), но это необязательно.

При использовании подзапросов в предикатах, основанных на реляционных операторах, надо убедиться, что использовали подзапрос, который будет выдавать одну и только одну строку вывода. Если вы используете подзапрос, который не выводит никаких значений вообще, команда не потерпит неудачи, но основной запрос не выведет никаких значений.

Подзапросы, которые не производят никакого вывода (или нулевой вывод), вынуждают рассматривать предикат ни как верный, ни как неверный, а как неизвестный. Однако, неизвестный предикат имеет тот же самый эффект, что и неверный: никакие строки не выбираются основным запросом.

Это плохая стратегия, чтобы делать что-нибудь подобное следующему:

```
SELECT *
FROM Orders
WHERE snum = (SELECT snum
              FROM Salespeople
              WHERE city = 'Barcelona');
```

Поскольку мы имеем только одного продавца в Barcelona – Rifkin, то подзапрос будет выбирать одиночное значение `snum` и, следовательно, будет принят. Но это – только в данном случае. Большинство SQL баз данных имеют многочисленных пользователей, и если другой пользователь добавит нового продавца из Barcelona в таблицу, подзапрос выберет два значения, и ваша команда потерпит неудачу.

В некоторых случаях можно использовать `DISTINCT`, чтобы вынудить подзапрос генерировать одиночное значение. Например, надо найти все Заказы для тех продавцов, которые обслуживают Hoffman (`snum = 2001`). Имеется один способ, чтобы сделать это:

```
SELECT *
FROM Orders
WHERE snum = (SELECT DISTINCT snum
              FROM Orders
              WHERE cnum = 2001);
```

Подзапрос установит, что значение поля `snum` совпало с Hoffman – 1001, и затем основной запрос выделил все Заказы с этим значением `snum` из таблицы Заказов (не разбирая, относятся они к Hoffman или нет). Так как каждый заказчик назначен к одному и только этому продавцу, мы знаем, что каждая строка в таблице Заказов с данным значением `snum` должна иметь такое же значение `snum`. Однако, так как там может быть любое число таких строк, подзапрос мог бы вывести много (хотя и идентичных) значений `snum` для данного поля `snum`. Аргумент `DISTINCT` предотвращает это. Если наш подзапрос возвратит более одного значения, это будет указывать на ошибку в данных.

Альтернативный подход должен заключаться в том, чтобы ссылаться к таблице Заказчиков, а не к таблице Заказов в подзапросе. Так как поле `snum` – это первичный ключ (таблицы Заказчика, запрос выбирающий его, должен произвести только одно значение. Это рационально только если вы как пользователь имеете доступ к таблице Заказов, но не к таблице Заказчиков. В этом случае вы можете использовать решение, которое мы показали выше.

Один тип функций, который автоматически может производить одиночное значение для любого числа строк – агрегатная функция. Любой запрос, использующий одиночную функцию агрегата без предложения `GROUP BY`, будет выбирать одиночное значение для использования в основном предикате. Например, надо увидеть все Заказы, имеющие сумму приобретений выше средней на 4-е Октября:

```
SELECT *
FROM Orders
WHERE amt > (SELECT AVG (amt)
             FROM Orders
             WHERE odate = 10/04/2016);
```

Сгруппированные агрегатные функции, которые являются агрегатными функциями, определенными в терминах предложения GROUP BY, могут производить многочисленные значения. Они, следовательно, не позволительны в подзапросах такого характера. Даже если GROUP BY и HAVING используются таким способом, что только одна группа выводится с помощью подзапроса, команда будет отклонена в принципе. В этом случае надо использовать одиночную агрегатную функцию с предложением WHERE, что устранил нежелательные группы. Например, следующий запрос находит среднее значение комиссионных продавца в Лондоне

```
SELECT AVG (comm)
FROM Salespeople
GROUP BY city
HAVING city = 'London';
```

не может использоваться в подзапросе! Во всяком случае, это не лучший способ формировать запрос. Другим способом может быть

```
SELECT AVG (comm)
FROM Salespeople
WHERE city = 'London';
```

Можно использовать подзапросы, которые производят любое число строк, если вы используете специальный оператор IN (операторы BETWEEN, LIKE, и IS NULL не могут использоваться с подзапросами). Когда используется IN с подзапросом, SQL формирует этот набор из вывода подзапроса. Мы можем, следовательно, использовать IN и найти все атрибуты таблицы Заказов для продавца в Лондоне:

```
SELECT *
FROM Orders
WHERE snum IN (SELECT snum
                FROM Salespeople
                WHERE city = 'London');
```

В ситуации подобно этой, подзапрос более прост для пользователя, чтобы понимать его, и более прост для компьютера, чтобы его выполнить, чем использовать объединение:

```
SELECT onum, amt, odate, cnum, Orders.snum
FROM Orders, Salespeople
WHERE Orders.snum = Salespeople.snum
AND Salespeople.city = 'London';
```

Хотя это и произведет тот же самый вывод, что и в примере с подзапросом, SQL должен будет просмотреть каждую возможную комбинацию строк из двух таблиц и проверить их снова по составному предикату.

Предположим, мы хотим знать комиссионные всех продавцов обслуживающих заказчиков в Лондоне:

```
SELECT comm
FROM Salespeople
WHERE snum IN (SELECT snum
                FROM Customers
                WHERE city = 'London');
```

Выводимыми для этого запроса являются значения комиссионных продавца Peel (snum = 1001), который имеет обоих заказчиков в Лондоне. Это – только для данного случая. Нет никакой причины, чтобы некоторые заказчики в Лондоне не могли быть назначенными к кому-то еще. Следовательно, IN – это наиболее логичная форма чтобы использовать ее в запросе.

Префикс таблицы для поля city необязателен в предыдущем примере, несмотря на возможную неоднозначность между полями city таблицы Заказчика и таблицы Продавцов.

SQL всегда ищет первое поле в таблице, обозначенной в предложении FROM текущего подзапроса. Если поле с данным именем там не найдено, проверяются внешние запросы. В вышеупомянутом примере, "city" в предложении WHERE означает, что имеется ссылка к Customer.city (поле city таблицы Заказчиков).

Можно использовать выражение, основанное на столбце, а не просто сам столбец, в предложении SELECT подзапроса. Это может быть выполнено или с помощью реляционных операторов, или с IN. Например, следующий запрос использует реляционный оператор = :

```
SELECT *
FROM Customers
WHERE cnum = (SELECT snum + 1000
              FROM Salespeople
              WHERE sname = 'Serres');
```

Он находит всех заказчиков, чье значение поля cnum равно 1000, выше поля snum Serres. Предполагаем, что столбец sname не имеет никаких двойных значений, иначе подзапрос может произвести многочисленные значения. Когда поля snum и cnum не имеют такого простого функционального значения как, например, первичный ключ, что не всегда хорошо, запрос типа вышеупомянутого невероятно полезен.

Можно также использовать подзапросы внутри предложения HAVING. Эти подзапросы могут использовать свои собственные агрегатные функции, если они не производят многочисленных значений или использовать GROUP BY или HAVING. Следующий запрос является этому примером:

```
SELECT rating, COUNT (DISTINCT cnum)
FROM Customers
GROUP BY rating
HAVING rating > (SELECT AVG (rating)
                 FROM Customers
                 WHERE city = 'San Jose');
```

Эта команда подсчитывает заказчиков с рейтингами выше среднего в San Jose. Так как имеются другие оценки отличные от 300, они должны быть выведены с числом номеров заказчиков, которые имели этот рейтинг.

Соотнесенный подзапрос

Соотнесенный подзапрос – один из большого количества тонких понятий в SQL из-за сложности в его оценке.

Например, имеется один способ найти всех заказчиков в Заказах на 3-е Октября:

```
SELECT *
FROM Customers outer
WHERE 10/03/2016 IN (SELECT odate
                    FROM Orders inner
                    WHERE outer.cnum = inner.cnum);
```

В вышеупомянутом примере, "внутренний" (inner) и "внешний" (outer), это псевдонимы. Мы выбрали эти имена для большей ясности; они отсылают к значениям внутренних и внешних запросов, соответственно. Так как значение в поле cnum внешнего запроса меняется, внутренний запрос должен выполняться отдельно для каждой строки внешнего запроса. Строка внешнего запроса, для которого внутренний запрос каждый раз будет выполнен, называется текущей строкой-кандидатом. Следовательно, процедура оценки выполняемой соотнесенным подзапросом – это:

- выбрать строку из таблицы именованной во внешнем запросе. Это будет текущая строка-кандидат;
- сохранить значения из этой строки-кандидата в псевдониме с именем в предложении FROM внешнего запроса;
- выполнить подзапрос. Везде, где псевдоним, данный для внешнего запроса, найден (в этом случае "внешний"), использовать значение для текущей строки-кандидата. Использование значения из строки-кандидата внешнего запроса в подзапросе называется — *внешней ссылкой*;
- оценить предикат внешнего запроса на основе результатов подзапроса выполняемого в шаге 3. Он определяет, выбирается ли строка-кандидат для вывода;
- повторить процедуру для следующей строки-кандидата таблицы, и так далее пока все строки таблицы не будут проверены.

Предположим, что мы хотим видеть имена и номера всех продавцов, которые имеют более одного заказчика. Следующий запрос выполнит это:

```
SELECT snum, sname
FROM Salespeople main
WHERE 1 < (SELECT COUNT (*)
          FROM Customers
          WHERE snum = main.snum);
```

Предложение FROM подзапроса в этом примере не использует псевдоним. При отсутствии имени таблицы или префикса псевдонима, SQL может для начала принять, что любое поле выводится из таблицы с именем, указанным в предложении FROM текущего запроса. Если поле с этим именем отсутствует (в нашем случае — cnum) в той таблице, SQL будет проверять внешние запросы. Именно поэтому, префикс имени таблицы обычно необходим в соотнесенных подзапросах для отмены этого предположения.

Можно использовать соотнесенный подзапрос, основанный на той же самой таблице, что и основной запрос. Это даст вам возможность извлечь определенные сложные формы произведенной информации. Например, надо найти все Заказы со значениями сумм приобретений выше среднего для их заказчиков:

```
SELECT *
FROM Orders outer
WHERE amt > (SELECT AVG (amt)
             FROM Orders inner
             WHERE inner.cnum = outer.cnum);
```

Также как предложение HAVING может брать подзапросы, оно может брать и соотнесенные подзапросы. Когда используете соотнесенный подзапрос в предложении HAVING, то надо ограничивать внешние ссылки к позициям, которые могли бы непосредственно использоваться в самом предложении HAVING. Предложение HAVING может использовать только агрегатные функции, которые указаны в их предложении SELECT или поля используемые в их предложении GROUP BY. Они являются только внешними ссылками, которые вы можете делать. Все это потому, что предикат предложения HAVING оценивается для каждой группы из внешнего запроса, а не для каждой строки. Следовательно, подзапрос будет выполняться один раз для каждой группы выведенной из внешнего запроса, а не для каждой строки.

Например, надо суммировать значения сумм приобретений покупок из таблицы Заказов, сгруппировав их по датам, удалив все даты, где бы сумма (SUM) не была, по крайней мере, на 2000.00 выше максимальной (MAX) суммы:

```
Select odate, SUM (amt)
FROM orders a
GROUP BY odate
HAVING SUM (amt) > (SELECT 2000.00 + MAX (amt)
                    FROM Orders b
                    WHERE a.odate = b.odate);
```

Подзапрос вычисляет значение MAX для всех строк с той же самой датой, что и у текущей агрегатной группы основного запроса. Это должно быть выполнено, как и ранее, с использованием предложения WHERE. Сам подзапрос не должен использовать предложения GROUP BY или HAVING.

Использование оператора EXISTS, ANY, ALL.

EXISTS – это оператор, который производит верное или неверное значение, другими словами, выражение Буля. Он берет подзапрос как аргумент и оценивает его как верный, если тот производит любой вывод, или как неверный, если тот не делает этого. Этим он отличается от других операторов предиката, в которых он не может быть неизвестным. Например, можно извлечь некоторые данные из таблицы Заказчиков, если, и только если, один или более заказчиков в этой таблице находятся в San Jose:

```
SELECT cnum, cname, city
FROM Customers
WHERE EXISTS (SELECT *
              FROM Customers
              WHERE city = 'San Jose');
```

Внутренний запрос выбирает все данные для всех заказчиков в San Jose. Оператор EXISTS во внешнем предикате отмечает, что некоторый вывод был произведен подзапросом, и поскольку выражение EXISTS было полным предикатом, делает предикат верным. Подзапрос

(не соотнесенный) был выполнен только один раз для всего внешнего запроса, и, следовательно, имеет одно значение во всех случаях.

В соотнесенном подзапросе предложение EXISTS оценивается отдельно для каждой строки таблицы, имя которой указано во внешнем запросе, точно также как и другие операторы предиката, когда используется соотнесенный подзапрос. Это дает возможность использовать EXISTS как верный предикат, который генерирует различные ответы для каждой строки таблицы, указанной в основном запросе. Следовательно, информация из внутреннего запроса будет сохранена, если выведена непосредственно, когда вы используете EXISTS таким способом. Например, мы можем вывести продавцов, которые имеют многочисленных заказчиков:

```
SELECT DISTINCT snum
FROM Customers outer
WHERE EXISTS (SELECT *
              FROM Customers inner
              WHERE inner.snum = outer.snum AND
                    inner.cnum <> outer.cnum);
```

Для каждой строки-кандидата внешнего запроса, внутренний запрос находит строки, которые совпадают со значением поля snum (которое имел продавец), но не со значением поля cnum (соответствующего другим заказчикам). Если любые такие строки найдены внутренним запросом, это означает, что имеются два разных заказчика, обслуживаемых текущим продавцом (т.е. продавцом заказчика в текущей строке-кандидате из внешнего запроса). Поэтому предикат EXISTS верен для текущей строки, и номер продавца – поле (snum) таблицы, указанной во внешнем запросе, будет выведен. Если был DISTINCT не указан, каждый из этих продавцов будет выбран один раз для каждого заказчика, к которому он назначен.

Самым простым способом для использования и вероятно наиболее часто используется с EXISTS — это оператор NOT. Один из способов, которым можно найти всех продавцов только с одним заказчиком, будет состоять в том, чтобы инвертировать наш предыдущий пример.

```
SELECT DISTINCT snum
FROM Customers outer
WHERE NOT EXISTS (SELECT *
                  FROM Customers inner
                  WHERE inner.snum = outer.snum AND
                        inner.cnum <> outer.cnum);
```

cnum

1003
1004
1007

Одна вещь, которую EXISTS не может сделать – взять функцию агрегата в подзапросе. Это имеет значение. Если функция агрегата находит любые строки для операций с ними, EXISTS верен, не взирая на то, что это — значение функции; если же агрегатная функция не находит никаких строк, EXISTS неправилен.

Попытка использовать агрегаты с EXISTS таким способом, вероятно, покажет, что проблема неверно решалась от начала до конца.

Конечно, подзапрос в предикате EXISTS может также использовать один или более из его собственных подзапросов. Они могут иметь любой из различных типов, которые мы видели (или который мы будем видеть). Такие подзапросы, и любые другие в них, позволяют использовать агрегаты, если нет другой причины по которой они не могут быть использованы.

Можно вкладывать два или более подзапроса в одиночный запрос, и даже один внутри другого. Имеется запрос, который извлекает строки всех продавцов, которые имеют заказчиков с больше чем одним текущим Заказом. Это не обязательно самое простое решение этой проблемы, но оно предназначено, скорее, показать улучшенную логику SQL. Вывод этой информации связывает все три наши типовых таблицы:

```

SELECT *
FROM Salespeople first
WHERE EXISTS (SELECT *
              FROM Customers second
              WHERE first.snum = second.snum AND
                    1 < (SELECT COUNT (*)
                       FROM Orders
                       WHERE Orders.cnum = second.cnum));

```

Разберём вышеупомянутый запрос примерно так:

Берем каждую строку таблицы Продавцов как строку-кандидат (внешний запрос) и выполняем подзапросы. Для каждой строки-кандидата из внешнего запроса берем в соответствие каждую строку из таблицы Заказчиков (средний запрос). Если текущая строка заказчиков не совпадает с текущей строкой продавца (т.е. если `first.snum <> second.snum`), предикат среднего запроса неправилен. Всякий раз, когда мы находим заказчика в среднем запросе, который совпадает с продавцом во внешнем запросе, мы должны рассматривать сам внутренний запрос, чтобы определить, будет ли наш средний предикат запроса верен. Внутренний запрос считает число Заказов текущего заказчика (из среднего запроса). Если это число больше 1, предикат среднего запроса верен, и строки выбираются. Это делает EXISTS предикат внешнего запроса верным для текущей строки продавца, и означает, что, по крайней мере, один из текущих заказчиков продавца имеет более одного заказа.

Специальные операторы ANY или SOME

Операторы SOME и ANY — взаимозаменяемы везде и там где мы используем ANY, SOME будет работать точно так же. Различие в терминологии состоит в том, чтобы позволить людям использовать тот термин, который наиболее однозначен. Имеется новый способ нахождения продавца с заказчиками, размещенными в их городах:

```

SELECT *
FROM Salespeople
WHERE city = ANY (SELECT city
                  FROM Customers);

```

Оператор ANY берет все значения выведенные подзапросом, (для этого случая — это все значения `city` в таблице Заказчиков), и оценивает их как верные, если любой (ANY) из их равняется значению города текущей строки внешнего запроса.

Это означает, что подзапрос должен выбирать значения такого же типа, как и те, которые сравниваются в основном предикате. В этом его отличие от EXISTS, который просто определяет, производит ли подзапрос результаты или нет, и фактически не использует эти результаты.

Оператор ANY может использовать другие реляционные операторы, кроме равенства (=), и, таким образом, делать сравнения, которые являются выше возможностей IN. Например, можно найти всех продавцов с их заказчиками, имена которых следуют в алфавитном порядке за именами продавцов.

```

SELECT *
FROM Salespeople
WHERE sname < ANY (SELECT cname
                   FROM Customers);

```

Обратите внимание, что это является основным эквивалентом следующему запросу с EXISTS:

```

SELECT *
FROM Salespeople outer
WHERE EXISTS (SELECT *
              FROM Customers inner
              WHERE outer.sname < inner.cname);

```

Любой запрос, который может быть сформулирован с ANY, мог быть также сформулирован с EXISTS, хотя наоборот будет неверно. Строго говоря, вариант с EXISTS не абсолютно идентичен вариантам с ANY или с ALL из-за различия в том, как обрабатываются

пустые (NULL) значения. Тем не менее, с технической точки зрения, вы могли бы делать это без ANY и ALL, если бы вы стали очень находчивы в использовании EXISTS (и IS NULL).

Подзапросы ANY или ALL могут выполняться один раз и иметь вывод, используемый чтобы определять предикат для каждой строки основного запроса. EXISTS, с другой стороны, берет соотношенный подзапрос, который требует, чтобы весь подзапрос повторно выполнялся для каждой строки основного запроса.

С помощью ALL, предикат является верным, если каждое значение выбранное подзапросом удовлетворяет условию в предикате внешнего запроса. Если мы хотим пересмотреть наш предыдущий пример, чтобы вывести только тех заказчиков, чьи оценки, фактически, выше, чем у каждого заказчика в Риме, можно ввести следующее:

```
SELECT *
FROM Customers
WHERE rating > ALL (SELECT rating
                    FROM Customers
                    WHERE city = 'Rome');
```

Этот оператор проверяет значения оценки всех заказчиков в Риме. Затем он находит заказчиков с оценкой большей, чем у любого из заказчиков в Риме. Самая высокая оценка в Риме — у Giovanni (200). Следовательно, выбираются только значения выше этих 200.

Как и в случае с ANY, мы можем использовать EXISTS для производства альтернативной формулировки такого же запроса:

```
SELECT *
FROM Customers outer
WHERE NOT EXISTS (SELECT *
                  FROM Customers inner
                  WHERE outer.rating <= inner.rating
                  AND inner.city = 'Rome');
```

ALL используется в основном с неравенствами, а не с равенствами, так как значение может быть "равным для всех" результатом подзапроса только если все результаты, фактически, идентичны. Рассмотрим следующий запрос:

```
SELECT *
FROM Customers
WHERE rating = ALL (SELECT rating
                   FROM Customers
                   WHERE city = 'Rome');
```

Эта команда допустима, но с этими данными мы не получим никакого вывода. Только в единственном случае вывод будет выдан этим запросом — если все значения оценки в San Jose окажутся идентичными. В этом случае, можно сказать следующее:

```
SELECT *
FROM Customers
WHERE rating = (SELECT DISTINCT rating
               FROM Customers
               WHERE city = 'Rome');
```

Основное различие в том, что эта последняя команда должна потерпеть неудачу, если подзапрос выведет много значений, в то время как вариант с ALL просто не даст никакого вывода. В общем, не самая удачная идея использовать запросы, которые работают только в определенных ситуациях, подобно этой. Так как база данных будет постоянно меняться, это неудачный способ, чтобы узнать о ее содержании.

Однако, ALL может более эффективно использоваться с неравенствами, то есть с оператором "<>".

Очевидно, если подзапрос возвращает много различных значений, как это обычно бывает, ни одно отдельное значение не может быть равно им всем в обычном смысле. В SQL, выражение <> ALL — в действительности соответствует "не равен любому" результату подзапроса. Другими словами, предикат верен, если данное значение не найдено среди результатов подзапроса. Следовательно, наш предыдущий пример противоположен по смыслу этому примеру:

```

SELECT *
FROM Customers
WHERE rating <> ALL (SELECT rating
                     FROM Customers
                     WHERE city = 'San Jose');

```

Вышеупомянутый подзапрос выбирает все оценки для города San Jose. Он выводит набор из двух значений: 200 (для Liu) и 300 (для Cisneros). Затем, основной запрос, выбирает все строки, с оценкой, не совпадающей ни с одной из них — другими словами все строки с оценкой 100.

Можно сформулировать тот же самый запрос, используя оператор NOT IN:

```

SELECT*
FROM Customers
WHERE rating NOT IN (SELECT rating
                    FROM Customers
                    WHERE city = 'San Jose');

```

Можно также использовать оператор ANY:

```

SELECT *
FROM Customers
WHERE NOT rating = ANY (SELECT rating
                       FROM Customers
                       WHERE city = 'San Jose');

```

Вывод будет одинаков для всех трех условий.